

КРАТКИЕ ОТВЕТЫ НА ЭКЗАМЕНАЦИОННЫЕ БИЛЕТЫ

Курс: Разработка приложений средствами Python / Проектирование ИСП (Устный ответ)

Билет №1

1. История создания языка Python. Сферы применения:

Создан Гвидо ван Россумом в 1991 году как читаемый и простой язык. Назван в честь шоу «Монти Пайтон». Ветвь Python 3 (2008) убрала обратную совместимость, исправив архитектурные ошибки. **Сферы:** ИИ/Data Science (библиотеки Pandas, PyTorch), веб-разработка (Django, FastAPI), автоматизация, парсинг и тестирование (PyTest).

2. Термин API. Возможности применения API:

API (интерфейс программирования приложений) — готовый набор функций и классов, позволяющий разным программам взаимодействовать. **Применение:** интеграция сторонних сервисов (оплата, авторизация через соцсети, карты, погода) и связь бэкенда с фронтендом.

Билет №2

1. Командная строка Python Shell и аргументы:

Python Shell (REPL) — интерактивная консоль для выполнения кода строка за строкой (запуск через python). ****Аргументы**** передаются скрипту при старте из терминала и считываются через список `sys.argv` (где `[0]` — имя файла), либо парсятся модулем `argparse`.

2. Технология написания бота на Python:

Бот отправляет HTTP-запросы к API платформы (например, Telegram). Получение данных идет через **Polling** (циклический опрос сервера) или **Webhooks** (сервер сам присылает уведомления). Пишется на библиотеках `aiogram` или `telebot` с использованием декораторов-хэндлеров.

Билет №3

1. Типы данных языка Python:

Python имеет динамическую сильную типизацию. Основные типы: `int` (целые), `float` (дробные), `str` (строки), `bool` (`True/False`). Коллекции: `list` (изменяемый список), `dict` (словарь), `tuple` (неизменяемый кортеж), `set` (множество) и специальный тип `None`.

2. Кроссплатформенные инструментальные средства:

Обеспечивают работу одного кода на Windows, Mac, Linux, Android и iOS. Главные инструменты на Python: фреймворк **Kivy** (и надстройка **KivyMD**), фреймворк **Flet** (на базе Flutter). Для сборки мобильных пакетов под Android применяется утилита **Buildozer**.

Билет №4

1. Работа с переменными в языке Python:

Переменная — именованная ссылка на объект в памяти. Создается через знак `=`, тип определяется автоматически. Имя может содержать буквы, цифры, `_`, не должно начинаться с цифры и совпадать с ключевыми словами. Рекомендуемый стиль — `snake_case`.

2. Фреймворк Kivy, язык KV и виджеты:

Kivy — графическая библиотека для кроссплатформенных UI. **Язык KV** — декларативная разметка для отделения дизайна от логики Python. **Виджеты** — строительные блоки UI: текстовые метки (`Label`), кнопки (`Button`), поля ввода (`TextInput`).

Билет №5

1. Встроенные функции языка Python:

Доступны глобально без импорта. Примеры: ввод/вывод (`print()`, `input()`), приведение типов (`int()`, `str()`, `list()`), расчеты и агрегаторы (`len()`, `abs()`, `max()`, `min()`, `sum()`), генерация последовательностей (`range()`).

2. Зарезервированные слова и выражения в языке KV:

Служат для управления логикой разметки: `self` (ссылка на текущий виджет), `root` (ссылка на корневой виджет текущего правила), `app` (доступ к экземпляру главного класса приложения на Python) и атрибут `id` для уникальной маркировки виджета.

Билет №6

1. Основные операторы языка Python:

Арифметические (+, -, *, /, целочисленное //, остаток %), сравнения (==, !=, >, <), логические (and, or, not), присваивания (=, +=), проверки членства (in) и тождественности (is).

2. Компоновка приложения методом соглашения имен:

Kivy автоматически связывает Python-код приложения с файлом разметки KV по имени класса. Если класс называется `TestApp(App)`, Kivy отбросит суффикс `App`, переведет имя в нижний регистр и автоматически подключит файл `test.kv` из этой же папки.

Билет №7

1. Методы работы со строками и числами:

Для чисел используют базовую арифметику или функции модуля `math` (`math.sqrt()`, `math.ceil()`). Строки неизменяемы, их методы: `.upper()/lower()` (регистр), `.split()` (разбиение), `.join()` (сборка из списка), `.replace()` (замена).

2. Виджеты в Kivy:

Делятся на две категории: **UX-виджеты** (элементы интерфейса для контента и ввода: `Label`, `Button`, `TextInput`, `Image`) и **Макеты (Layouts)** — невидимые контейнеры для позиционирования дочерних виджетов (`BoxLayout`, `GridLayout`).

Билет №8

1. Использование операторов языка Python:

Операторы формируют выражения. Из-за полиморфизма их поведение зависит от типов данных: оператор + сложит числа (`2+3=5`), но склеит строки (`"a"+"b"="аб"`). При написании выражений важно соблюдать приоритет операторов (например, умножение над сложением).

2. Задание размеров и положения виджетов в окне Kivy:

Используются адаптивные свойства `size_hint` (доля от размера контейнера от 0 до 1) и `pos_hint` (процентная привязка координат). Для фиксации размера в пикселях/dp нужно сбросить подсказку (`size_hint: None, None`) и задать точные значения через `size`.

Билет №9

1. Условные конструкции:

Обеспечивают ветвление кода с помощью `if` (если), `elif` (иначе если), `else` (иначе). Структура держится на отступах. Существует краткая запись (тернарный оператор: `x = A if condition else B`) и конструкция сопоставления шаблонов `match/case`.

2. Задание виджетам цвета фона:

У базовых виджетов (как `Label`) нет свойства цвета фона, его рисуют на холсте `canvas.before` (объекты `Color` и `Rectangle`). У кнопок `Button` свойство `background_color` накладывается как фильтр, поэтому предварительно сбрасывают текстуру через `background_normal: ''`.

Билет №10

1. Циклические алгоритмы:

Используются для повторения задач. Цикл `while` выполняется, пока истинно условие. Цикл `for` последовательно перебирает элементы коллекций или диапазонов `range()`. Оператор `break` мгновенно прерывает цикл, а `continue` перескакивает на следующую итерацию.

2. Обработка событий виджетов:

События — реакция на действия пользователя. В коде KV обработчики пишутся прямо в свойствах с приставкой `on_` (например, `on_press: app.method()`). В Python-коде функции привязываются к событиям виджета через метод `.bind(on_release=callback)`.

Билет №11

1. Списки, словари и методы работы с ними:

Список (list) — изменяемая упорядоченная последовательность. Методы: `.append()` (добавить), `.remove()` (удалить), `.sort()` (сортировка). **Словарь (dict)** — пары "ключ: значение". Методы: `.get()` (безопасное чтение), `.keys()`, `.values()`, `.items()`.

2. Дерево виджетов как основа интерфейса:

UI в Kivy устроен как иерархическое дерево: во главе стоит один Корневой виджет (`Root`), возвращаемый приложением, в него вложены макеты-контейнеры, а внутри них — конечные элементы. Дерево задает правила распределения размеров и путь прохождения событий касания.

Билет №12

1. Объекты — кортеж, множества:

Кортеж (tuple) — неизменяемый список в круглых скобках `(1, 2)`. Защищает данные от перезаписи и экономит память. **Множество (set)** — неупорядоченная группа уникальных элементов `{1, 2}`. Исключает дубликаты, поддерживает пересечение и объединение.

2. Виджеты для позиционирования элементов (Layouts):

Контейнеры управления версткой в Kivy: `BoxLayout` (размещает виджеты цепочкой по вертикали/горизонтали), `GridLayout` (строит регулярную таблицу по столбцам/строкам), `AnchorLayout` (прибивает к краям/центру), `FloatLayout` (абсолютные координаты).

Билет №13

1. Понятие функции и методы создания:

Функция — именованный изолированный кусок кода для решения подзадачи, созданный для избежания дублирования (DRY). Объявляется через ключевое слово `def` имя(параметры) :. Возвращает результат в программу с помощью инструкции `return`.

2. Виджет Carousel для пролистывания слайдов:

Контейнер в Kivu для создания экранов-слайдов, которые пользователь листает горизонтальными или вертикальными свайпами. Поддерживает заикливание картинок/экранов (`loop: True`) и программный сдвиг через методы `.load_next()` и `.load_previous()`.

Билет №14

1. Передача аргументов. Лямбда-функции:

Аргументы бывают **позиционными** (передаются строго по порядку) и **именованными** (с указанием имени параметра). Параметрам можно задавать значения по умолчанию. `*args` и `**kwargs` принимают переменное число аргументов. **Лямбда** (`lambda`) — короткая анонимная однострочная функция.

2. Задание цвета фона виджету-контейнеру:

Макеты (например, `BoxLayout`) по умолчанию прозрачны. Их цвет рисуется через `canvas.before` в коде KV. Чтобы фон не ломался при изменении размеров окна, геометрию прямоугольника привязывают к самому контейнеру: `pos: self.pos` и `size: self.size`.

Билет №15

1. Анонимные функции в функциях высшего порядка:

Функции высшего порядка принимают другие функции как аргументы или возвращают их. Лямбды идеальны для них, так как пишутся в одну строку ради разового действия. Применяются внутри встроенных функций фильтрации и трансформации: `map()`, `filter()`, `sorted()`.

2. Индексация элементов в дереве виджетов:

Каждый контейнер хранит список своих прямых потомков в свойстве-списке `children`. Индексация в нем идет в обратном порядке относительно их добавления: самый последний добавленный элемент на экране будет доступен по первому индексу `children[0]`.

Билет №16 / №17

1. Синхронные и асинхронные функции:

Синхронные функции выполняются строго последовательно и блокируют поток выполнения (интерфейс виснет на долгих задачах). **Асинхронные** функции (`async def`) работают через event loop библиотеки `asyncio` и не блокируют программу, засыпая на время I/O через `await`.

2. Классы Screen и ScreenManager:

Организуют многоэкранные интерфейсы. Класс `Screen` — заготовка под отдельный экран приложения со своим дизайном и уникальной строковой меткой `name`. Класс `ScreenManager` — контейнер-менеджер, который переключает экраны через свойство `current`.

Билет №18

1. Возможности подключаемых библиотек языка Python:

Экосистема делится на **стандартные модули** (идут с Python: `os` — система, `math` — математика, `json`, `sys`) и **сторонние** (скачиваются через `pip` из репозитория PyPI, например `requests`, `kivy`, `pandas`). Подключаются в код инструкцией `import`.

2. Класс `Window` для формирования окна приложения:

Класс `Window` (из `kivy.core.window`) — это программный интерфейс к системному окну ОС. Позволяет менять размеры окна (`Window.size`), задавать цвет подложки всего приложения, перехватывать нажатия физической клавиатуры и управлять виртуальной клавиатурой.

Билет №19 / №20

1. Модули и функции библиотек `time` и `random`:

Модуль `time`: `time.time()` (Unix-время в секундах), `time.sleep(c)` (заморозка потока на 'c' секунд). Модуль `random`: `random.randint(a, b)` (случайное целое число в отрезке), `random.choice(коллекция)` (выбор одного случайного элемента из списка).

2. Структура приложений на KivyMD:

KivyMD строит интерфейсы по правилам Google Material Design. Главный класс наследуют от `MdApp` (вместо `App`), а виджеты заменяются на аналоги с префиксом MD (`MdLabel`, `MdRaisedButton`). В главный класс встроен менеджер тем и палитра `theme_cls`.

Билет №21

1. Открытие и сохранение файлов разных форматов:

Для текста (`.txt`): функция `open()` с менеджером контекста `with` (методы `.read()`, `.write()`). Для данных (`.json`): встроенный модуль `json` (методы `json.load()` для чтения и `json.dump()` для записи). Для таблиц — модуль `csv` или библиотека `pandas`.

2. Структура многоэкранных приложений на `ScreenManager`:

Корнем интерфейса ставится `ScreenManager`, в него регистрируются дочерние `Screen`. Каждый экран имеет доступ к менеджеру через свойство `root.manager` или `self.manager`. Смена экрана происходит строчкой `manager.current = 'имя_экрана'`.

Билет №22

1. Понятие объекта. Поля и методы объекта:

Объект — экземпляр класса, сущность в памяти, имеющая состояние и поведение. **Поля (атрибуты)** — это переменные объекта, описывающие его свойства (создаются через `self.имя` в конструкторе `__init__`). **Методы** — функции внутри класса, принимающие первым аргументом `self`.

2. Стили KivyMD для задания цвета элементам:

Цвета элементов привязаны к дизайн-системе Material Design. Вместо случайных RGB-кодов используются свойства цвета KivyMD (например, `md_bg_color`) и ссылки на системную палитру приложения, такие как `app.theme_cls.primary_color` или текстовые имена цветов ("Red").

Билет №23

1. Свойства ООП (инкапсуляция, наследование, полиморфизм):

Инкапсуляция: сокрытие внутренней логики и данных (в Python реализуется соглашением через `_` и `__`).
****Наследование:**** перенос полей и методов родительского класса в дочерний. ****Полиморфизм:**** единый интерфейс работы с разными типами объектов (Duck Typing).

2. Темы KivyMD для настройки цветовых оттенков:

Глобальный тон настраивается через объект `theme_cls` в главном классе `MdApp`. Свойство `theme_style` меняет тему со светлой на темную ("Light" / "Dark"). Свойства `primary_palette` и `accent_palette` задают основные цвета UI (например, "Teal").

Билет №24

1. Работа магических методов:

Магические методы (dunder) — специальные встроенные методы с двойным подчеркиванием (`__метод__`), вызываемые интерпретатором автоматически. Примеры: `__init__` (конструктор объекта), `__str__` (описание объекта для `print()`), `__repr__` (отладочный вывод).

2. Компоненты KivyMD для позиционирования элементов:

Это Material-версии контейнеров Kivy с встроенной поддержкой теней и скруглений углов: `MdBoxLayout` (линейные ряды), `MdGridLayout` (табличная сетка), `MdFloatLayout` (свободные слои) и карточки `MdCard`, выступающие самостоятельным красивым контейнером.

Билет №25

1. Переопределение действий магических методов:

Переопределение (перегрузка операторов) позволяет обучить свои объекты стандартным математическим и логическим операциям. Переписывая метод `__add__`, мы меняем поведение знака `+` для объектов, переписывая `__eq__` — оператора сравнения `==`.

2. Компоненты пользовательского интерфейса KivyMD:

Базовые Material-элементы: `MdLabel` (текст со шрифтовыми стилями), кнопки `MdRaisedButton` (кнопка с заливкой и тенью) и `MdIconButton` (иконка), текстовое поле `MdTextField` (со всплывающей подсказкой), верхняя панель навигации `MdTopAppBar`.

Билет №26

1. Диаграммы деятельности. Рекомендации по построению:

Диаграмма деятельности (Activity) — UML-схема, отражающая пошаговый бизнес-процесс или алгоритм логики программы (блок-схема). ****Рекомендации:**** строго один старт и финиш; разбивка ролей по "плавательным дорожкам" (Swimlanes); декомпозиция тяжелых кусков.

2. Гибкие методологии проектирования:

Agile — семейство подходов к созданию ПО, строящихся на коротких итерациях, быстрой адаптации к изменениям и тесной связи разработчиков с заказчиком. Работающий продукт ставится выше бумажной документации, а люди — выше процессов.

Билет №27

1. Диаграммы последовательности. Рекомендации по построению:

Диаграмма последовательности (Sequence) — UML-схема взаимодействия объектов, упорядоченная по времени. Показывает обмен сообщениями (вызовами). **Рекомендации:** время идет строго сверху вниз; объекты строятся слева направо по ходу включения; мелкий код опускается.

2. Agile методология:

Подход, при котором проект бьется на небольшие циклы (итерации от 1 до 4 недель). На финише каждой итерации команда сдает рабочий инкремент (новую версию ПО с готовыми функциями), что защищает проект от рисков сделать ненужный или нерабочий продукт.

Билет №28

1. Требования к UI. Принципы создания GUI:

Интерфейс обязан быть интуитивным, отзывчивым, лаконичным и согласованным. **Принципы:** консистентность (единый стиль во всем приложении), контроль (наличие опций отмены и шага назад), защита от ошибок пользователя (валидация полей и блокировка кнопок).

2. Scrum методология:

Фреймворк внутри Agile, делящий работу на **Спринты** (2-4 недели). **Роли:** Product Owner (представитель заказчика), Scrum Master (координатор процесса), команда разработки. **События:** Планирование спринта, Daily Standup (15 минут каждый день), Демо и Ретроспектива.

Билет №29

1. Спецификация, синтаксис и стиль языка:

Спецификация — документ со стандартами языка. **Синтаксис** — грамматические правила написания (в Python блоки отделяются отступами). **Стиль** — правила оформления кода для улучшения читаемости. Для Python эталоном стиля является стандарт **PEP 8**.

2. Lean методология:

Концепция "бережливой" разработки ПО, пришедшая из Toyota. Главная цель — **устранение любых потерь** (ненужного функционала, простоев, дефектов, бюрократии). Базируется на быстрой доставке MVP клиенту, обучении команды и оптимизации всей системы.

Билет №30

1. Разработка графического интерфейса пользователя:

Проектирование GUI состоит из стадий: анализ целевой аудитории, создание прототипов и схем экранов (в Figma), разделение логики и представления (архитектура MVC / язык KV), программная верстка интерфейса и финальное юзабилити-тестирование.

2. Kanban методология:

Гибкая методология Agile без жестких ролей и спринтов. Процесс идет непрерывно и визуализируется на **Канбан-доске** с колонками ("Сделать", "В работе", "Готово"). Поток регулируется **WIP-лимитами** (ограничением на число одновременно выполняемых задач).